Tabla de contenidos

1. *Client-side script*
    a. Ejemplo consumiendo una API (método `get`)
    b. Retorno de una API
    c. Uso de `post`
    d. Autenticación en *headers*
    e. Caso aplicado: API de GITHUB
2. Resumiendo

Client-side script

Primero, nos enfocaremos en el punto de vista del cliente al momento de utilizar *webservices*, por lo que veremos cómo efectuar *requests* a un servidor que mantiene un servicio web. En Python, la librería `requests` nos permite interactuar con servicios disponibles en algún *web service*. La librería, además, integra los métodos para serialización en JSON.

Para instalar la librería `requests`, en cualquier terminal debes correr el comando `python3 -m pip install requests`. Otra opción, es ejecutar la siguiente celda, que instalará la librería en el mismo entorno en que estés corriendo este jupyter (de todas maneras recomendamos instalarlo desde la consola).

Recuerda que si `python3` no funciona, probar con `python`, `py` o `py3`.

```
!python3 -m pip install requests
```

```
Requirement already satisfied: requests in
./.venv/lib/python3.12/site-packages (2.32.5)
Requirement already satisfied: charset_normalizer<4,>=2 in
./.venv/lib/python3.12/site-packages (from requests) (3.4.5)
Requirement already satisfied: idna<4,>=2.5 in
./.venv/lib/python3.12/site-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
./.venv/lib/python3.12/site-packages (from requests) (2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in
./.venv/lib/python3.12/site-packages (from requests) (2026.2.25)
```

Para generar una petición mediante `GET` usamos el método `get(url)` que recibe por argumento el llamado al recurso.

```
import requests

# Esta url contiene la dirección del web service
# y los parámetros que se requiren para la consulta
url =
```

```
'https://api.github.com/repos/IIC2233/Syllabus/contents/Actividades'
respuesta = requests.get(url)
```

Luego podemos usar `.status_code` para saber el código de estado de la consulta.

```
print(f'Status: {respuesta.status_code}')
```

```
Status: 200
```

```
import requests

# Esta url contiene la dirección del web service
# y los parámetros que se requiren para la consulta
url =
'https://api.github.com/repos/IIC2233/Syllabus/contents/Actividades'
respuesta = requests.get(url)

# El output de esta respuesta particular
# puede ser transformado con .json() a un diccionario o una lista
# en este caso, fue a una lista
data = respuesta.json()
for file in data:
    #print(file)
    print("-" * 20)
    print(file["name"], file["url"])
```

```
--------------------
.keep
https://api.github.com/repos/IIC2233/Syllabus/contents/Actividades/.ke
ep?ref=main
--------------------
AC1
https://api.github.com/repos/IIC2233/Syllabus/contents/Actividades/
AC1?ref=main
--------------------
AC2
https://api.github.com/repos/IIC2233/Syllabus/contents/Actividades/
AC2?ref=main
--------------------
AC3
https://api.github.com/repos/IIC2233/Syllabus/contents/Actividades/
AC3?ref=main
--------------------
AC4
https://api.github.com/repos/IIC2233/Syllabus/contents/Actividades/
AC4?ref=main
--------------------
AC5
https://api.github.com/repos/IIC2233/Syllabus/contents/Actividades/
AC5?ref=main
```

```
--------------------
AC6
https://api.github.com/repos/IIC2233/Syllabus/contents/Actividades/
AC6?ref=main
--------------------
AC7
https://api.github.com/repos/IIC2233/Syllabus/contents/Actividades/
AC7?ref=main
--------------------
ACBonus
https://api.github.com/repos/IIC2233/Syllabus/contents/Actividades/
ACBonus?ref=main
```

Ejemplo consumiendo una API (método `get`)

A continuación vamos a experimentar con la API de *DummyJSON*. Esta nos entrega distintos tipo información falsa en formato JSON. Por ejemplo, usuarios, productos, recetas, etc.

En primer lugar, todas las APIs tendrán una **URL BASE**, que consiste en un *link* de prefijo que toda consulta debe tener. Para el caso de *DummyJSON*, esta URL BASE es `"https://dummyjson.com/"`. Por lo tanto, cualquier solicitud que hagamos, debe partir con ese prefijo.

Luego, tenemos los ***endpoint***. Para efectos del curso, podemos considerar estos ***endpoints*** como las diferentes rutas que dispone la API para hacer consultas. Para el caso de *DummyJSON*, algunos ***endpoints*** son

- `products/`: nos retornará información de productos.
- `recipes/`: nos retornará información de recetas.
- `carts/` nos retornará información de carros de compra.
- `image/`: nos retornará imágenes personalizadas.

Por lo tanto, para consultar esta api, deberemos combinar la **URL BASE** con un ***endpoint***. De este modo, a URL final sería `"https://dummyjson.com/products"`, `"https://dummyjson.com/recipes"`, entre otros. Siempre que visitemos una API, esta nos detallará los diferentes *endpoints* que tiene.

Vamos a probar 2 ***endpoints*** de *DummyJSON*.

```python
import requests


BASE = "https://dummyjson.com/"
endpoint_1 = "carts/"

respuesta = requests.get(BASE + endpoint_1)

# imprimir el status_code
print(respuesta.status_code)
```

```
200
```

Ahora vamos a ver el contenido de la consulta

```
data = respuesta.json()
print("Llaves: ", data.keys())

Llaves:  dict_keys(['carts', 'total', 'skip', 'limit'])

data["total"]

208

import requests

BASE = "https://dummyjson.com/"
endpoint_1 = "carts/"

respuesta = requests.get(BASE + endpoint_1)

data = respuesta.json()

data['carts'][1]

{'id': 2,
 'products': [{'id': 86,
   'title': 'Man Short Sleeve Shirt',
   'price': 19.99,
   'quantity': 5,
   'total': 99.94999999999999,
   'discountPercentage': 6.83,
   'discountedTotal': 93.12,
   'thumbnail': 'https://cdn.dummyjson.com/product-images/mens-
shirts/man-short-sleeve-shirt/thumbnail.webp'},
  {'id': 104,
   'title': 'Apple iPhone Charger',
   'price': 19.99,
   'quantity': 2,
   'total': 39.98,
   'discountPercentage': 18.52,
   'discountedTotal': 32.58,
   'thumbnail': 'https://cdn.dummyjson.com/product-images/mobile-
accessories/apple-iphone-charger/thumbnail.webp'}],
 'total': 139.93,
 'discountedTotal': 125.7,
 'userId': 2,
 'totalProducts': 2,
 'totalQuantity': 7}
```

Cómo podemos notar, nos entregó la información completa de una receta.

Ahora vamos a hacer una consulta al endpoint de `"products/"`.

```
endpoint_2 = "products/"

respuesta = requests.get(BASE + endpoint_2)

data = respuesta.json()
[product['title'] for product in data['products'][:2]]
```

```
['Essence Mascara Lash Princess', 'Eyeshadow Palette with Mirror']
```

Parámetros de una consulta

Independientemente del tipo de consulta (`GET`, `POST`, ...), cada ***endpoint*** puede tener sus parámetros para personalizar la consulta.

Por un lado, *DummyJSON* tiene algunos parámetros que sirve para cualquier ***endpoint***. Algunos de estos son:

- `"limit"`: nos permite indicar la cantidad de elementos a obtener. Por defecto es 30, pero ahora vamos a cambiarlo a 3.
- `"select"`: nos permite indicar qué información queremos recibir. Por defecto nos entrega toda la información, lo cambiaremos para que solo nos entregue el nombre de los ítems.

Por otro lado, el ***endpoint*** de `"recipes/search/"` nos permite buscar recetas con alguna palabra en específico. Para esto, tendremos que utilizar el parámetro `"q"`. Busquemos las recetas que contengan el texto `"Chicken"`:

```
# Los parámetros los podemos pasar de dos formas:

# 1. Directamente en el URL de la request. En este caso será todo lo
que esté
# después del '?' serán parámetros, donde cada uno de ellos está
separado por un '&'

endpoint_3 = "recipes/search?limit={}&select={}&q={}"
respuesta = requests.get(BASE + endpoint_3.format(5, 'name',
'tomato'))

data = respuesta.json()
data['recipes']
```

```
[{'id': 7, 'name': 'Tomato Basil Bruschetta'},
 {'id': 46, 'name': 'Pesto Pasta with Cherry Tomatoes'}]
```

```
# 2. Como un argumento del método. Para esto haremos un diccionario
# con cada parámetro y su valor.

endpoint_3 = "recipes/search/"
parametros = {"limit": 3, "select": "name", "q": "chicken"}
```

```
respuesta = requests.get(BASE + endpoint_3, params=parametros)

data = respuesta.json()
data['recipes']
```

```
[{'id': 4, 'name': 'Chicken Alfredo Pasta'},
 {'id': 5, 'name': 'Mango Salsa Chicken'},
 {'id': 11, 'name': 'Chicken Biryani'}]
```

Adicionalmente, en algunas ocasiones podemos entregar argumentos por medio del *endpoint*. Un ejemplo es el *endpoint* de `"image/"`, que nos permite crear imágenes de portes, colores y textos personalizados, para esto tendremos que adaptar nuestro *endpoint* y utilizar parámetros. El formato del *endpoint* es el siguiente:

```
"image/SIZE/BACKGROUND_COLOR/FONT_COLOR"
```

donde:

- `SIZE` es el porte de la imagen (ancho x alto) en pixeles. Por ejemplo: `150x200`
- `BACKGROUND_COLOR` es el color del fondo en formato HEX. Por ejemplo: `008080`. Por defecto este valor es gris.
- `FONT_COLOR` es el color de la fuente en formato HEX. Por ejemplo: `ffffff`. Por defecto este valor es negro.

Además, con los parámetros se puede indicar lo siguiente:

- `text`: contenido del texto a poner en la imagen. Por defecto son las dimensiones de la imagen.
- `fontFamily`: fuente utilizada para escribir el texto. El conjunto de fuentes a utilizar se indica en la documentación del *endpoint*.

Puedes revisar con mayor detalle los parámetros que presenta este *endpoint* en el siguiente link.

```
consulta = "image/"
porte = "400x200/"
color_fondo = "82944b/"
color_fuente = "d4d2bc"

endpoint_4 = consulta + porte + color_fondo + color_fuente

parametros = {
    "text": "Todos somos Pepa lovers",
    "fontFamily": "ubuntu"
}

respuesta = requests.get(BASE + endpoint_4, params=parametros)
link_imagen = respuesta.url

from IPython.display import display, HTML, Image
from os import path
```

```
with open(path.join('data', 'img_personalizada.png'), 'wb') as file:
    file.write(respuesta.content)

display(Image(path.join('data', 'img_personalizada.png')))
```



Te invitamos a cambiar los valores del *endpoint* y los párametros para crear tus propias imágenes.

Retorno de una API

Como vimos antes, existen requests que nos retornan la información en un JSON:

```
import requests
from json import JSONDecodeError


BASE = "https://dummyjson.com/"
endpoint_1 = "recipes/"
parametros = {"limit": 10, "select": "name"}

respuesta = requests.get(BASE + endpoint_1, params=parametros)
respuesta.json()
```

```
{'recipes': [{'id': 1, 'name': 'Classic Margherita Pizza'},
  {'id': 2, 'name': 'Vegetarian Stir-Fry'},
  {'id': 3, 'name': 'Chocolate Chip Cookies'},
  {'id': 4, 'name': 'Chicken Alfredo Pasta'},
  {'id': 5, 'name': 'Mango Salsa Chicken'},
  {'id': 6, 'name': 'Quinoa Salad with Avocado'},
  {'id': 7, 'name': 'Tomato Basil Bruschetta'},
```

```
  {'id': 8, 'name': 'Beef and Broccoli Stir-Fry'},
  {'id': 9, 'name': 'Caprese Salad'},
  {'id': 10, 'name': 'Shrimp Scampi Pasta'}],
 'total': 50,
 'skip': 0,
 'limit': 10}
```

Pero también hay algunas que entregan la información en otro formato:

```
consulta = "image/"
porte = "400x200/"
color_fondo = "82944b/"
color_fuente = "d4d2bc"

endpoint_4 = consulta + porte + color_fondo + color_fuente

parametros = {
    "text": "Todos somos Pepa lovers",
    "fontFamily": "ubuntu"
}

respuesta = requests.get(BASE + endpoint_4, params=parametros)
try:
    respuesta.json()
except JSONDecodeError:
    content_type = respuesta.headers['Content-Type']
    print('El contenido de la response no se encuentra en formato
JSON, ' \
          f'sino que su contenido es "{content_type}"')

respuesta.content
```

```
El contenido de la response no se encuentra en formato JSON, sino que
su contenido es "image/png"
```

```
b'\x89PNG\r\n\x1a\n\x00\x00\x00\rIHDR\x00\x00\x01\x90\x00\x00\x00\xc8\
x08\x06\x00\x00\x00\xc6\x15\xb7\xe2\x00\x00\x00\tpHYs\x00\x00\x03\xe8\
x00\x00\x03\xe8\x01\xb5{Rk\x00\x00\x14&IDATx\x9c\xed\x9c\xf7{\x1c\xd5\
x15\x86\xf9\xff\xa4\xd0BI\xa3\xa4\x10\x02\x01\x12\x1ez\x08\x0f\xc9\
x017l\xdc-W\x15[\x96mY\x92eYV\x97{\xef\x85\xfc?7\xcfw\xc7W\x9e]\xedJ.\
x04\x9dC\xde\x1f\xbe\x07vvvg\xe6=\xdf\x9eo\xee\xbd#?c\xdb;\x12\x82\
x01\x1e\xc0\x03x\x00\x0f\xd8c2x\x06\xd3`\x1a<\x80\x07\xf0\x00\x1e\
xb0\'`@\x80`\x1c\x9a\x07\x1e\xc0\x03x \x11 \x98\x80F\x80\x07\xf0\x00\
x1eH?\x15\x03F \x98\x8d\x86\x83\x07\xf0\x00\x1eH\x04\x08&\xa0\x11\xe0\
x01<\x80\x07\x12#\x10L@#\xc0\x03x\x00\x0f$\xcf\x0c\x98\xc2rP\x04\x04\
x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\
x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\
xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\
x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\
x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\
```

x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022
@\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\
x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\
xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\
x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\
x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\
x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\
xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\
x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\
x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\
xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\
x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\
x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\
xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\
x1e\x88\xe8\x01\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\
x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\
x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\
xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\
x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\
x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\
x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\
x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\
xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\
x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\
x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\
x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\
xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\
x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\
x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\
xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\
x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\
x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\
xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\
x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\
x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\
x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022
@\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\
x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\
xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\
x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\
x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\
x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\
xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\
x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00\xf9?\xd6\xea\x9d/\xa4\x91\
x895\xa9g\xf0\xc3\x15?\x17\xf4\xd30\xd8s\xe8\xbdtrfs\xb2\xed\x9d0\
xc7w\xe9i\x19\x10 \x0eM\xb4f\xd7\x0bi\xd7\xc0;\xcb\xea\x9b\x1d\xcf>\
xd5q6\xec\xfbU\xfa\xcf\xfd\xf944\xfe\xed\x8a_sd\xad\xd9\xf5b\xda\xd1\
xffV\x0e\xe2\xde\xa3\x1f\xe5&\xbd\xa5\xe7\xf5\xf4\xcd\x8e_\xac\xf8\
xb95k|vs\xae9\x01\xb2\xf2\xb5\xb0\x9f\x81\x08\x10\x87\xda\xda\xf7\x87\

```
xfc#_Nkv\xfd\xf2\xa9\x8eC\x80\xfc8\xf5\x1a\x9d\\\xdb\xb2>\xb7n\x9dH\
xfd#\x9f\xaf\xb8\x9f\xea"@V\xbe\x06\xf63\x12\x01\xe2P\xab\xba\x9e\xcb!
RtlbMnH\x07G\xfe\xd1\xb0\xfdi\xef"\t\x90\x1f7@v\x1e|\'}\xbf\xef7is\
xcf\xeb\xa9w\xe8\xe3t\xe5\xeaP\xde~`\xf8\xd3\x15\xf7T\x11\x01\xb2\
xf25\xb0\x9f\x91\x08\x90\x00:x\xec\xcb\xdc\x88v\r\xbc\xdb\xf2\xfdU;\
x9fO\xddG\xfe\x9e\x0e\x1f\xff*O\xa3\xac\xde\xf9|\xcb\xfd\xbe\xedz6\
xed=\xf4~\x1a8\xf6e\xdaw\xe4\x83\xb4\xb9\xf7\x8d\xb6SX\x9a\x82\xd1q\
xb5\xef\x96\xbe7[~\xdfw{^M}G?N\x87\xc7\xbeN}G?\xca\xcds\xa9\xeb\xd8\
xd2\xfbF\xea\x1f\xfe,\x1d\x1a\xfd*\xed;\xf2\xb7\xb4n\xd7KM\xfbt\xa6\
xae\xfe?\xa7\x81\xd1\x7f\xa6\x03\xc3\x9f\xe7\x80[|\r\xcf\xa5m\xfb\xff\
x98\xaf\xb3\x7f\xe4\x8b\xd4s\xf4\xc3\x85\xfd\xd6\xefy%\xf5\x0e~\x94\
xfa\x8f}\xf1`\x8a\xaf\xf5yl\xee}=\x87\xf1\xc0\xe8\x97i[\xdf\xef\xdb\\\
xdb+\xa9o\xe8\x93|m\n\x83\x8d\xdd\xbf]6@\x9a\xf7\xd1\xf9\xdc\xbf;\
x9b.\\\x1ah<~\x8d\xed\xd6\x16lW\xef|1u\xf5\xbf\x9d\xa7\xc3t-\xba\xd6\
xb5\xbb\x17\x8f6UgM\x97\x89\x83\xaee\xf7\xc0\xbb\xb9\xc6O\x1a \xdb\
xf7\xff)\xb3\xd717u\xff\xae\xe1\xbd\x1d\x07\xdeJ\xdb\x0f\xbc\xb5\xe83\
x1b\xf6\xfd:\xb3\xd69\x97m\xaakfw\xfc\xab|~\xad\xa6\xf26\xf5\xbc\x96\
xf6\x1d\xfe \x1fK\x1e\xfa~\xefC\xeft\x1d|;\xb3\xa9\xae\xe9\xaf\xb9\
xe6+\xfd\x1bD\x1d-\x19\x10 \xc1\x03D?\xea\x9b7\xc7\xf2\xfb?\xdc{8u\
xa2F\\\xdfO\xcd\xed\xea\xb5\xe1\x86)\x96\xfbwg\x16\x05\x88~\xec\x13\
xb3[\x1e\xec3\xf7@\xf3ir~{\xfa\xb6\xeba#\xe8\x1e\xfc0\x7f^\xba}\xebd\
xdegbnk\xdbk86\xa1&;\x97\xee\xdd\x99Jw\xefL\xe4\xfd\xeb\xd3;j\x90g/\
xeco\xb8\x8e\x1f\xee\xcd\xa4\xc1\xb1\x7f7|\xcf\xf4\xa9\xaeESE\xf7\
xefN\xa7\xd93\xbb\xd3\x0f\xf7f\x1b\xb6\x9f>\xd7\xd3\xd4\xbc:\xd3\xf8\
xcc\xc6\x85k*\xc7\xd1wj\xd4W\xf6S\xb8\xdd\xbb;\x9d\x9b\x7f\xb9\xb6\
xa9S\xdb\x1f;@\xa4\xeb\xd7Gs}\xca\xf1OLo\xc8\xc7\xd7\xb5\x95\xf3\x1d\
x9d\xfc\xae\xe13\xaa_\xf35\x8a\x9b\x9aj}?5\xe7\xfc\xde\x83s\xd5\xff_\
xb9z4?\x1c\xf18\x01\xa2\x1b\x90\xb9\xb3{\x9a\xd8\xcf\xa7\xe3S\xdf-\
xec7szg\xaeu\x9d\x9341\xb7%\x1f[\xebv\x85\xdd\xdd;\x93\x0b\xe7\xa5\
xff^\xbc<\xd00\xdd:6\xb5~\xd1\xf5)\x04\xb5\xa6w\xea\\w~}\xe7\xf6\xc9\\
W\xfd\xff\xd66!\x8f:V\x9c\x01\x01\x128@\xd4to\xdd\x1eO7n\x8e\xe5;D5K\
xddE^\xbf~,\xff\x00\xd7\xedyy\xa1q\xe9G\xac\x06\xa0\xbb=\xdd\xb9\xea\
xeeX\xc1\xd1\x1c GOX\xde\xa6F\xa7\x1f\xbd\xee,K\x83\xd4\x13[\xf5\
xc6X5\x86\xaaq\xe8;u7\xda\xea\xfc\xd7\xef\xad\x16\xeb\xf5\xf4Oi\xe8\
xba\xc3\xaf\xdfU\xcf\x9d\xd9\x9d\x1b\xabF\x06jR\x1a\xdd\xcc\x9f\xa9\
x9a\x9a\x9aR\xd9O\xcd^\x8d\xa5|Vw\xf3\xba^\xed\xa7\xbb^\xdd\xfd\xea\
xbc\'\xe7\xb6\xe6m\x1a\x99\x95\xcf\x1e>\xfeu\xde\xa6&\xaa\xcf\xeb\
xdcGN\xae\xca\xdb\xaafY\xed\xa7\xa0\xbdt\xe5\xc8\xc2]\xf5\xaa%\xaem\
xa9\x00\xd15($.\\:\x98_\xeb\xee\xbe\xe2\xb8:_\xa3\xa4`\xaa\xa6\xbf\
xfe\xd2\x10 g/\xf4\xe5sTh\xab\xa6\x1a\xc5h\xbf=\x87\xdf_\xd8OAQj\xac\
xe6\xab\xd1\x92\xf692\xf6\xf5c\x05\xc8\xf8\xcc\xa6\xbcM\xb5\xd7w\xae\
xdd\xfdR\x9a\x9c\xdf\xb60m\xaa}4\x1a\xca\xc7?\xf4^\xed\xfb:\xb3\xf7N\
x9f\xef\xcd\xafu\xfd\n\x8ds\x17\x0fd\x7f\xe9}\xf9M\x9f\x1b\x9b\xda\
xb0\xe8xe\xe4\xa8\x87\x10\xc4Zu\xd6\xf6zPj$\xe4\xf1a\x04\xd4\x91\x19\
x10 \x81\x03\xa44$M\xe3\xd4\xb7\xab\xc9h{\xb9{\xd74M\xf5#^\xbf\xcc\
x1aHg\x0e\xa4\xcbW\x06\x17M\xff\xa8\x81\xdd\xbd3\xb5p\x07\xaaQ\xc4\
xf9\x8b\xfdm\xa7\x89\xeaR\x93on\xd2u\xa9\xf1(<\xd4\xf4\xeb\xdb\xd5T\
xcaq\xea\x01\xa2&U\xdf\xaf\x84\x9e\xa6\xc8\x1a\xbf\xb3\x1ez\x9dy$\
xa0ph\x9e\xbeQ\xc3\xd3w*(\xf4\xfa\xf6\xad\xf1t1O;u>\xd1\x1a\x88\xa6\
```

xfc4UW\xd6@\xd4D\xb5\x9f\x82N\xc1T\xff\xdeu\xbb_^4\nQ\x80\xcc\x9f\xdd\
xdbp\x0c\x05\xaeF!\x1a\xa5\xb5;\x0f\xd5B7\x0e\x1a-<j\x80(l5\x1a:}\xae\
xbba?\x05\x97n\x12t\xce\xd5~/\xe4\x1b\x10\x8d\xe0\xca>\x9aJ\xac7|]C\
xab =wa\xff\xc2\xf7\xd4\x03\xa4>\xa2\x95\xf6\x0f}\x92\xb7k\x1ar\xa5\
x7fs\xa8\xe3\x91\x18\x10 \x81\x03d\xfa\xd4\x8e\x07Oc\xbd\xd8\xf4\xe3\
x7f6OAhZ\xa2~\xf7\xa89\xe7\xa5\x02dc\xf7k-\x83F*w\xea\xd5\xe2}\xc7\
xc2\xc2\xfe\xa5\xcb\x87s\xb3l\x9e\xdahTg:{\xbe/\xef\x7f\xea\xdc\xbe\
xbc\x0eS\x0f\x1e\xad\x9f\xe4\xc61T5\xda\xba\xd4\xd8t\x17_\x1a^\xab\
x00\xd1\xe7\x9a\xf9\xa89\x95Q\x8f^k\x8e=\x8f>j\r\xb09\x80\xca\x1c\xff\
xf0\x83k\xbdtu0O\xad\xb4[SZ\xee)\xac\x1b7\x8e\xa7\x03#\x9f\xe5}4\x9a\
xc8SL\xd7\x86r\x03\xadK\xd7Wo\xfa\xad\x02D:s\xbe77\xfb\xc2Nw\xe6\x1aa\
xe9\x9a\xf4\xf9\xf1\xe9\x8dy\xcaMSw\x8f\x1a Z\xbf\xd0k\x8d^\x9a\xf7-\
xa3\x10\x85\x9c^\xeb\x9c\xaa\xe9\xb8\xce\x85iI\x9d\x8fF,z\xadp\xd3tV\
xf3\xf5)\xb4\xb5_\xf9\\\xbb\x00\xd1\xf7\x88\x99\xf6\xd5\xd4\x98F^+\
xfd\xdbC\x1dK2 @\x02\x07\x88\x02BA\xd1\xeaNYM\xf6\xcc\xf9\x9e\x86\x91\
x8a\x9a\xc5R\x01\xa2\xb9f\xbdV\x03m\xfe\xbe2=R_[Q\xf3\xd2\xf4L^w\xb9=\
x9e\x17\xe6\xdb]\x83\x9a\x85\x16\xcf\xaf\xdd\x18\xc9\xfb_\xbe:\x98\
x17\xf1\xf5\x9e\xd6B\xb4M\xc1\xd2\xfc\xb9\xb2\xe6Q\x02\xaaU\x80\x94\
xa9\x8f\xc6\xe9\x95\x8e\xccFM\xb0,\x9ck\x1f5\xbd\xe6ch\xb1\xb6\x99\
xaf\xc2V#\x1fmWS^\xea\x8f-K\x80\x88\xb3\xceE\xd3\x89\xd5]xg\xc3\x08\
xa2\n\x95\xd1\xdc\xe0\x9b5:\xb9n\xd9\x00),\xb4\xa8\xac\x10\xd1\xba\
x8f^\xeb\x0e_\xebVz\xadQ\xc2\xe3\x04\xc8\xde\xc3\x1fTk\x10\xc3U\xd0\
xd5U\xd6*\xca\xc3\x11e\x84Pq\xea\xcc\xb5\xd44c\xd9\xff\xfc\xa5\xfe\
x85\xe3\xb7R\t\xe2v\x01R\x82V5\xd2H\xaaZ\xc7\xean\xf1\xb0\x052\'\x0c\
x08\x10\x07Ex\xd2\x0091\xf3}\x9b\xb9\xf7W\x1a\x16\xb5\xd5\xe8\xeb\xf3\
xd9\xed\x02\xa4\x9aN\x99\xcb\x0b\xe6\x8b\x1aO^|\x9eo\xb9\x16\xa0\xa7\
x7f\xd4L\xd4\xd8\x9bGC\xcd\xd2\\\xbd\x9a\x95\xd61\xb4\x86R_\x0cV\x03n\
xde\xff\xc2\xe5C\xb9\x81\x97\xd7O\x1a :/\xbdn5\xbdS\x1a\xe5\xa6\x9e\
xc6\'\x8f$\x85\xee\xb5k\xc3\xf9\x98\xed\x1a\xd9R\x8b\xe8\x0f\xd5\x99\
xaf\xb9U04\xab]\x80(\xac\x0b\x0b\x8d\x96\xaa@|\xb8.%\xe9n\xffq\x02\
xa4Lo\xd6\xd7\xb7\x8at\x0e\x1a\r\x94\xf0\xd6\x7fu\xfc\xa9\xb9m\xf9F\
xa2\x99\xb9\xd6s4\x9aZz4\xbat\x80\xd4\xa7/uNy\xd48\xbdx\xd4\x88:\\0 @\
x02\x07\x88\x1aw\xabi\x99\xd2\xd0\xca\xe2\xb3\xe6\xb9\xb5~\xa1\xe6R\
x7f\xdc\xb2\xd5\xdf\x81\x9c\xbfx 7\xba\xfac\x95\n$\xadE\\\xce\xf3\xf7\
xad\xcf\xf1\xc8\xd8\xbf\xaau\x886\x8f\xfc6Kw\xcb\x9a\xd3\xaf\x9a\xc5\
xf3\xf9\x8eSk\x06\xf5\xe6S\x9a\x94\xee\xae\x9f6@\xca\x14\x90\xee\x90\
xeb\x8d^\xd3&j\x8aW\xaf\x1dm\xbb\xe6\xa1\x91\x93\xbe\x7f\xdb\xfej\xfa\
xee\xc9\x02\xa4#/6k\x8aG\x8fH/\x17 z\x1a\xa9\xbeM\xd39\xf5p\xd7(\xa7<\
xbd\xf48\x01Rn:\xcazO\x19I\xdc\xbay\xa2\xe1\xa1\x06\xad[)\x0c\x9a\
x83L\xa3\x03\xf1\xd7\xc8C\xc7\xaaOE\x16\x9f\xca\x0bO\x1b EZ\xa4_j\xdd\
x07u\xac(\x03\x02$\xf4c\xbc\x9d\xf9\x8eZ\xef\xcd\x9e\xde\x9d\xa7\x99\
xca4\x87~\xe0\xf5\x1fw5M4\x97\x173u\xd7\xaag\xf4K3\xa9\x07\x88\x16F\
x15 \x9a\xeb\xd6T\x96\xde\xd3\xbc\xb4\x1a\xef\xc3\xe9\xab\xce\xdcX\
xb4.\xa2\x06\xa6\x86\xa1\x05W->\xb7zf_k+\n\x8c\xa1\xf1o\xf2y\xe8)$\
x1d\xa3\xde$\xf7\x0f\x7f\x9a\xcfO\x8b\xccZ\xfc\xd7\x82\xbb\x02F\xe7Q=\
xd1\xf3\xf4\x01\xa2\xbf\xb9\xd0w\xaaA\xeb\x1ct>j\x9ej\xea\xbbjOA\x89\
x9d\xde\xd7\x88MOni\x1f\x85L\xbb\xb5\x90G\r\x10\xad\x1fU\x8f1O\xe5\
xbbj\xd5K|5\x15Y\xafU\xf5\x18\xef\\f<x\xc2\xd2\xd8\xf4\x86\xfch\xec\
x9d\xdb\x93\x0b\x7f\x9f\xa1\xa7\xa5\xd4\xf45uX\xb8\xea|\x97[\x03Q\xbd\

x8a_\xca\xf9V\x0f]\xcc\xe5@\xd0w\xc9\x1f\xfa\x1e\xdd44\x8f\xca\xaa\
xb5\xa4\xea1h\x05k\xfd=\x85\xff\xc5\xcb\x87\xf2{\xba&\xd5Q\xebK\x1a\t\
xd7\xa7>\xdb\x05\x88F\xca\xdaW~\xd2\xf5\x9c\x9c\xd5~s\xd9g+\xfd\x1bD\
x1d-\x19\x10 \x01\xcc\xa1\xf9wM\xf7\xb4ZT\xd4\x94Pi\x84\xfaQ\xaa\x91\
xeb\x07\xd7\xea\x0f\xca\xd4`u7\xa7&\xa6\x06\xae\xf9x\xbd\xae?\x1aZ\
x1a\x9d\xe6\x9e\xb5OYK\xa9\x1f[\xcdKMVa\xa4\xe6\xab&\xaf\x86\xd1n\xf4\
xa1\xa7\xa3t\x1c\xed\xa7\xfdu\xdc\xeaQ\xda\x97\x16\x05\x81\xa6\xac4m\
xa2f9=\xdf\xb5h\xcaL\xd3\x1a\x9a\xf3\xafo\xd3#\xb0\xe2\xa3\x7f\x8f\
xaa\xf9\xc9\xb1\xe6?\x92T\x88h!\xbf\xfa\xdb\x89\x99|^\xf5\xcf\xa9\t\
xea\x91\xe2\x12,j\xd2j\xe4K\xfd-\x82\xa6\xdet\xfc\xf5{_]\xb6\x96Z\
xf7Q\xe8+$t\x9d7o\x1e\xcf\xc7\xab\x87\x93\xde\xd3hL5\xa8\x1a\xf9d>\
x87\xb2f\xb4\xc0\xb5\xef\xcd\xcc\xbd\xe2:\x93?\xa7\x00n\xb5\x86U$_h\
xc4\xaa\xda\xe9f\xa1lW\x80V\xa3\xcf\xd9\xec\x0f}\xaf\xfe\xd8\xaf\xd5w\
xe8\x11oM\xa7\xb5\x9a\xae\xcc\x8f}O\xacM\xd7\xae\x8f\xe4\xeb\xd3M\x85\
xd6F4b*\xfb(\x14\xc5\xab\xf9\xdfr+\xf5\xd7gT\x1f\x05\x9a\xae\xe5i\xff\
xcd7\xd4\xf1?c@\x80`0~`\xce<\xd0n\r\x04\xc1\xc0\x9c1 @\x1c\x14\x01\
xc1\x80\x00\xc1\x03\x16\x90\x01\x01\xe2\xa0\x08\x08\x06\x04\x08\x1e\
xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\
x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\
x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\
x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\
x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\
xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\
x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\
x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\
x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\
xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\
x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\
x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\
xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\
x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\
x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\
xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\
x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\
x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\
x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022
@\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\
x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\
xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\
x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\
x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\
x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\
xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\
x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\
x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\
xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\
x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\
x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\
xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\

```
x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\
x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\
x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022
@\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\
x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\
xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\
x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\
x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\
x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\
xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\
x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\
x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\
xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\
x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\
x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\
xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\x11\x10\x0c\xf0\x00\
x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022 @\x1c\x14\x01\xc1\
x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\x07, \x03\x02\xc4A\
x11\x10\x0c\xf0\x00\x1e\xb0\x80\x0c\x08\x10\x07E@0\xc0\x03x\xc0\x022
@\x1c\x14\x01\xc1\x00\x0f\xe0\x01\x0b\xc8\x80\x00qP\x04\x04\x03<\x80\
x07, \x83\xff\x02\xee\x98\x1b\x1e\x91\x08\xe7\x01\x00\x00\x00\x00IEND\
xaeB`\x82'
```

Uso de `post`

En el caso de la API con la que hemos estado haciendo estas pruebas, solo se ofrecen servicios para realizar consultas, lo que se puede llevar a cabo utilizando el método `GET` del protocolo HTTP. Sin embargo, muchas veces queremos crear recursos en nuestro servidor, como por ejemplo un comentario en Instagram, y para esto debemos utilizar el método `POST` del protocolo.

Siguiendo con el uso de `DummyJSON`, ahora utilizaremos la API para crear un nuevo producto. En la práctica no estará ocurriendo nada en el servidor, puesto que es solo una simulación, pero en la vida real uno esperaría que como respuesta a nuestra *request* se cree una entrada en la base de datos del servicio que estamos utilizando.

A diferencia del método `GET`, cuando utilizamos el método `POST` podemos enviar información a la API utilizando el parámetro `data`, al cual podemos pasarle un diccionario de Python con la información que queremos enviar.

```python
import requests


BASE = "https://dummyjson.com/"
endpoint_5 = "products/add"

# Los datos del producto que vamos a crear:
data = {
    'title': 'Comida de Pepa',
```

```
    'description': 'Alimento esencial de una tortuga',
    'price': 2233,
    'rating': 5,
}

respuesta = requests.post(BASE + endpoint_5, data=data)

# Vemos que obtenemos un código de que nuestro producto fue creado
print(respuesta.status_code)
print(respuesta.reason)
```

```
201
Created
```

```
# Esta API nos retorna un JSON con el mismo recurso creado, notese que
se le asignó un id al artículo
data = respuesta.json()
print(data.keys())
```

```
dict_keys(['id', 'title', 'price', 'rating', 'description'])
```

```
data
```

```
{'id': 195,
 'title': 'Comida de Pepa',
 'price': '2233',
 'rating': '5',
 'description': 'Alimento esencial de una tortuga'}
```

Autenticación en *headers*

Aparte del método `post`, tambien tenemos `.put()`, `.patch()` o `.delete()`. Generalmente, este tipo de requests repercuten en la modificación de una base de datos, y para lograr esto, muchas veces es necesario tener una autorización previa.

Es aquí que surge la necesidad de ocupar el parámetro `headers` que permite incluir información en la cabezara de la solicitud incuyendo, por ejemplo, un token especial de acceso que a la API le sirve para identificarte y verificar si tienes los permisos.

Utilizar este parámetro es análogo a `data`. Solo es necesario crear un diccionario e incluirlo en la *requests*. Las llaves del diccionario puede cambiar según la API que queramos acceder.

```
# Primero obtendremos una key (en este caso un token), para poder
# hacer consultas que necesitan autentificación.
url = 'https://dummyjson.com/auth/login'
body = {'username': 'emilys', 'password': 'emilyspass',
'expiresInMins': 30}

response = requests.post(url, data=body)
data = response.json()
```

```
TOKEN = 'Bearer ' + data['accessToken']
print('Token:', TOKEN)
```

```
Token: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm5hbWUiOiJlbWlse
XMiLCJlbWFpbCI6ImVtaWx5LmpvaG5zb25AeC5kdW1teWpzb24uY29tIiwiZmlyc3ROYW1
lIjoiRW1pbHkiLCJsYXN0TmFtZSI6IkpvaG5zb24iLCJnZW5kZXIiOiJmZW1hbGUiLCJpb
WFnZSI6Imh0dHBzOi8vZHVtbXlqc29uLmNvbS9pY29uL2VtaWx5cy8xMjgiLCJpYXQiOjE
3Nzc1NTg0MzEsImV4cCI6MTc3NzU2MDIzMX0.aRfS7E5C7_ffNVpzl0g9CXpXakpkAi-
vDLe8RTssMkg
```

```
# Ahora, utilizaremos el token obtenido en la celda anterior para
obtener
# información sobre el usuario correspondiente.

url = 'https://dummyjson.com/auth/me'
header = {'Authorization': TOKEN}

respuesta = requests.get(url, headers=header)
respuesta.status_code
```

```
200
```

```
respuesta.json()
```

```
{'id': 1,
 'firstName': 'Emily',
 'lastName': 'Johnson',
 'maidenName': 'Smith',
 'age': 29,
 'gender': 'female',
 'email': 'emily.johnson@x.dummyjson.com',
 'phone': '+81 965-431-3024',
 'username': 'emilys',
 'password': 'emilyspass',
 'birthDate': '1996-5-30',
 'image': 'https://dummyjson.com/icon/emilys/128',
 'bloodGroup': 'O-',
 'height': 193.24,
 'weight': 63.16,
 'eyeColor': 'Green',
 'hair': {'color': 'Brown', 'type': 'Curly'},
 'ip': '42.48.100.32',
 'address': {'address': '626 Main Street',
  'city': 'Phoenix',
  'state': 'Mississippi',
  'stateCode': 'MS',
  'postalCode': '29112',
  'coordinates': {'lat': -77.16213, 'lng': -92.084824},
  'country': 'United States'},
 'macAddress': '47:fa:41:18:ec:eb',
```

```
 'university': 'University of Wisconsin--Madison',
 'bank': {'cardExpire': '05/28',
  'cardNumber': '3693233511855044',
  'cardType': 'Diners Club International',
  'currency': 'GBP',
  'iban': 'GB74MH2UZLR9TRPHYNU8F8'},
 'company': {'department': 'Engineering',
  'name': 'Dooley, Kozey and Cronin',
  'title': 'Sales Manager',
  'address': {'address': '263 Tenth Street',
   'city': 'San Francisco',
   'state': 'Wisconsin',
   'stateCode': 'WI',
   'postalCode': '37657',
   'coordinates': {'lat': 71.814525, 'lng': -161.150263},
   'country': 'United States'}},
 'ein': '977-175',
 'ssn': '900-590-289',
 'userAgent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/96.0.4664.93
Safari/537.36',
 'crypto': {'coin': 'Bitcoin',
  'wallet': '0xb9fc2fe63b2a6c003f1c324c3bfa53259162181a',
  'network': 'Ethereum (ERC20)'},
 'role': 'admin'}
```

```
# Finalmente, comprobemos cómo se comporta la request cuando NO
entregamos el token.
url = 'https://dummyjson.com/auth/me'

respuesta = requests.get(url)

print(respuesta.status_code)
print(respuesta.reason)
```

```
401
Unauthorized
```

Caso aplicado: API de GITHUB

A continuación, te presentamos un caso aplicado para crear una *issue* en un repositorio de Github mediante su API. Las *GitHub Issues* son elementos que puede crear en un repositorio para planear, analizar y realizar un seguimiento del trabajo. Por ejemplo, aquí podemos observar como una librería de *Machine Learning* utiliza las *issues* para que los usuarios reporten información sobre la librería.

En este caso, para crear una *issue*, la API de Github pide que el `data` a enviar sea un diccionario en su forma de *string*, es decir, `json.dumps(data)`.

Además, para que este caso aplicado funcione correctamante, se requiere:

1. Completar `github_repo` con el nombre de tu repositorio de github dado en el curso. Por ejemplo: `"prueba-iic2233-2026-1"`
2. Completar `token` con un *access token* generado en Github. Puedes generar uno en esta página. Este *token* debe tener al menos el siguiente permiso para poder crear una issue: *repo: Full control of private repositories*.

```
import json
import requests

github_repo = 'prueba-iic2233-2026-1'
TOKEN = "ghp_token"

body = {
    'title': "Creando una issue con la API",
    'body': "Ahora tengo el poder para hacer issues desde Python! "
}

my_headers = {
    'Authorization': 'token ' + TOKEN,
    'Accept': 'application/vnd.github.v3+json'
}

url = f"https://api.github.com/repos/IIC2233/{github_repo}/issues"

respuesta = requests.post(url, data=json.dumps(body),
headers=my_headers)
respuesta.status_code
```

```
401
```

Si la celda anterior arrojó un `201`. Significa que el *token* utilizado y el `github_repo` estaban correctos, así que se creó la *issue*. Ejecuta la siguiente celda para ver el *link* a dicha issue.

```
if respuesta.status_code == 201:
    print("Issue creada exitosamente!")
    print("Puedes verla en:", respuesta.json()["html_url"])
else:
    print("Hubo un error al crear la issue. Status code:",
respuesta.status_code)
```

```
Hubo un error al crear la issue. Status code: 401
```

Te invitamos a consumir diferentes APIs desde Python para poner a prueba este contenidos. Algunas ideas de estudio pueden ser:

- Revisar los diferentes *endpoint* que tenía *DummyJSON* para inventar más información.
- Revisar la documentación de OMDb API para buscar películas.
- Usar la APi de pokemon para buscar información de tu pokemon favorito.

Ahora que conocemos los métodos principales de `ThreadPoolExecutor`, vamos a ver en la práctica la diferencia entre la forma **secuencial** y la forma **concurrente** de enviar requests, comparando el tiempo total de cada enfoque al obtener 10 recetas de la API de *DummyJSON*.

Resumiendo

Todas las APIs tienen:

- Una **URL BASE**, que consiste en un *link* de prefijo que toda consulta debe tener.
- Uno o más ***endpoint***, que -para efectos del curso- son las diferentes rutas que dispone la API para hacer consultas.

En lo que respecta a las consultas (*requests*):

- Independiente de su tipo (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`), podrán **utilizar los parámetros o el *body*** para entregar información adicional a la API.

- Dependiendo de su tipo (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`), puede **cambiar la forma en que se comporte el *endpoint*** (si es que se encuentra definido para más de un tipo de consulta).
- Se utilizarán los *headers* para enviar las **credenciales de autentificación**, en caso de que sean necesarias para realizar la consulta.

En lo que respecta a las respuestas (*responses*):

- Los datos asociados a la respuesta se encontrarán en el ***body*** y, dependiendo de la información transmitida, estos **podrán ser distintos tipos de datos** (HTML, JSON, imágenes, etc.)